Clue Selection Under Noise

Codenames spymaster, baseline model

1 Introduction

A clue is a clue word together with a number k. Whenever we give one, our teammate ranks the unrevealed board words by how well each fits the clue word, and chooses words from the top, stopping when they reach k words or when they choose a word that is not ours, whichever comes first.

Suppose we and our teammate order words by cosine similarities from the same embedding space. Then choosing the optimal clue is just a search over the clue vocabulary for the word with the longest starting run of our team's words in the list sorted by decreasing similarity. In this case, our teammate chooses only the words that we intend, and never chooses a word that is not ours. The search is a matter of enumerating the vocabulary, and its solution is usually not unique.

However, in practice, our perceived similarities are related to but different from our teammate's, which changes the algorithm by which we find the optimal clue. The problem becomes probabilistic rather than deterministic, and we now have to account for the possibility of our teammate choosing a neutral, opponent, or assassin word. This paper describes one algorithm for finding an optimal clue under these conditions.

2 Setup

Let A be the set of our team's board words and B the set of all other board words. For any fixed clue word, let z_x be its similarity to board word x. Throughout, i indexes A and w indexes B. Selecting a word of A is worth 1, and selecting $w \in B$ costs $c_w > 0$. We want to choose both parts of the clue.

We assume the guesser perceives board word x's similarity with the clue word as $\hat{z}_x = z_x + \varepsilon_x$, where the ε_x are independent $\mathcal{N}(0, \sigma^2)$. This models differences between our interpreted embeddings and the guesser's — higher σ means a larger gap between the two. We also assume the guesser selects unrevealed words in decreasing order of $\hat{z}$, stopping after k selections or upon selecting a word of B, whichever comes first.

In words, the guesser ranks the board the same way we do, perturbed by independent noise of a single scale, and then guesses greedily. σ is the only free parameter.

3 Reduction

Let

$$T = \max_{w \in B} \hat{z}_w, \qquad W^* = \arg\max_{w \in B} \hat{z}_w, \qquad N = \#\{\, i \in A \,:\, \hat{z}_i > T \,\}.$$

Every word ranked above T is in A, since T is the maximum of $\hat{z}$ over B. Thus the guesser's ordering begins with the N words counted above, and the $(N+1)$-th word is W^*. This means the guesser reveals $\min(k, N)$ words of A, and selects a word of B if and only if $N < k$, in which case that word is W^*.

Note that no word of B other than W^* can be selected, so the single random variable T accounts for all of B. Write φ and Φ for the density and the distribution function of a standard normal. Each $\hat{z}_w$ is $\mathcal{N}(z_w, \sigma^2)$, so

$$F_{\hat{z}_w}(t) = \Phi\Big(\frac{t - z_w}{\sigma}\Big), \qquad f_{\hat{z}_w}(t) = \sigma^{-1}\varphi\Big(\frac{t - z_w}{\sigma}\Big).$$

Since T is a maximum of independent variables,

$$F_T(t) \;=\; \prod_{w \in B} F_{\hat{z}_w}(t) \;=\; \prod_{w \in B} \Phi\Big(\frac{t - z_w}{\sigma}\Big).$$

4 Conditional distribution

T is a function of $(\varepsilon_w)_{w \in B}$ only, which is independent of $(\varepsilon_i)_{i \in A}$. Thus conditioning on $T = t$ does not change the distribution of the $\hat{z}_i$, and each $i \in A$ independently satisfies $\hat{z}_i > t$ with probability

$$p_i(t) \;=\; \Phi\Big(\frac{z_i - t}{\sigma}\Big).$$

Given $T = t$, the count N is therefore a sum of $|A|$ independent Bernoulli variables, the i-th with success probability $p_i(t)$.

5 Objective

From Section 3 the guesser reveals $\min(k, N)$ words of A, and selects W^* exactly when $N < k$. Each word of A is worth 1 and W^* costs c_{W^*}, so the value of announcing k is $G(k) - P(k)$, where

$$G(k) = \mathbb{E}\big[\min(k, N)\big], \qquad P(k) = \mathbb{E}\big[\, c_{W^*}\, \mathbf{1}\{N < k\}\,\big].$$

Since $\min(k, N) = \sum_{j=1}^{k} \mathbf{1}\{N \geq j\}$, taking expectations and conditioning on T gives

$$G(k) \;=\; \sum_{j=1}^{k} \int \Pr(N \geq j \mid T = t)\; dF_T(t).$$

Each $\Pr(N \geq j \mid T = t)$ follows from the Bernoulli sum of Section 4, and the integral is one-dimensional in t. Section 6 computes both. It remains to find P.

In P the cost c_{W^*} and the event $\{N < k\}$ both depend on the noise, but they become independent once T is fixed. To see this, note that T is a function of $(\varepsilon_w)_{w \in B}$ alone, so conditioning on $T = t$ constrains that family and leaves $(\varepsilon_i)_{i \in A}$ with its original distribution. Given $T = t$, the count $N = \#\{i \in A : z_i + \varepsilon_i > t\}$ is a function of $(\varepsilon_i)_{i \in A}$,

while the identity W^* is a function of $(\varepsilon_w)_{w\in B}$. The two families are independent, so N and W^* are conditionally independent given T, and the expectation splits:

$$P(k) \;=\; \int \Pr(N < k \mid T = t)\; \bar{c}(t)\; dF_T(t), \qquad \bar{c}(t) = \mathbb{E}\big[\, c_{W^*} \mid T = t \,\big].$$

The first factor is again a probability for the Bernoulli sum of Section 4. It remains to compute $\bar{c}(t)$, for which we need $\Pr(W^* = w \mid T = t)$.

Writing g_w for the joint density of $W^* = w$ and T at t,

$$\Pr(W^* = w \mid T = t) \;=\; \frac{g_w(t)}{f_T(t)}.$$

$W^* = w$ and $T = t$ means $\hat{z}_w = t$ and $\hat{z}_v < t$ for every other $v \in B$. The $\hat{z}$ are independent, so

$$g_w(t) \;=\; f_{\hat{z}_w}(t) \prod_{v \neq w} F_{\hat{z}_v}(t).$$

The events $\{W^* = w\}$, $w \in B$, are disjoint and exhaustive, so

$$f_T(t) \;=\; \sum_{w \in B} g_w(t).$$

From Section 3, $F_T(t) = \prod_{v\in B} F_{\hat{z}_v}(t)$, so the product over $v \neq w$ is $F_T(t) \big/ F_{\hat{z}_w}(t)$. Set

$$h_w(t) \;=\; \frac{f_{\hat{z}_w}(t)}{F_{\hat{z}_w}(t)} \;=\; \frac{\sigma^{-1}\varphi\big((t - z_w)/\sigma\big)}{\Phi\big((t - z_w)/\sigma\big)},$$

so that $g_w(t) = F_T(t)\, h_w(t)$ and $f_T(t) = F_T(t) \sum_v h_v(t)$. The $F_T(t)$ cancels in the ratio:

$$\Pr(W^* = w \mid T = t) \;=\; \frac{h_w(t)}{\sum_v h_v(t)}, \qquad \bar{c}(t) \;=\; \frac{\sum_w c_w\, h_w(t)}{\sum_w h_w(t)}.$$

We choose the clue maximizing the value,

$$\text{clue}^* \;=\; \operatorname*{arg\,max}_{\text{word},\; 1 \le k \le K} \big[\, G(k) - P(k) \,\big].$$

6 Computation

Both G and P have the form $\int \psi(t)\, dF_T(t)$ for a bounded ψ:

$$\psi_G(t) = \sum_{j=1}^{k} \Pr(N \ge j \mid T = t), \qquad \psi_P(t) = \Pr(N < k \mid T = t)\, \bar{c}(t).$$

Both are evaluated by numerical integration. Partition an interval $[t_0, t_M]$ into cells $[t_m, t_{m+1}]$, take ψ at the midpoint $\bar{t}_m = \frac{1}{2}(t_m + t_{m+1})$ of each, and weight it by the probability that T lands in that cell:

$$u_m \;=\; F_T(t_{m+1}) - F_T(t_m), \qquad \int \psi(t)\, dF_T(t) \;\approx\; \sum_m \psi(\bar{t}_m)\, u_m.$$

The u_m are exact, being differences of the product in Section 3, so the only error is the variation of ψ within a cell.

Take t_0 below every z_w and t_M above every z_w, each by several σ, so that $F_T(t_0)$ and $1 - F_T(t_M)$ are negligible and no mass falls outside the grid. The same grid and the same u_m serve both integrals, so one pass over the cells gives $G(k)$ and $P(k)$ together, and for every $k \le K$ at once.

Within a cell, $\bar{c}(\bar{t}_m)$ is immediate from its definition, at a cost of $|B|$ terms. The probabilities $\Pr(N \ge j \mid T = \bar{t}_m)$ follow from the standard recursion for a sum of independent Bernoulli variables. Letting $q_r^{(i)}$ be $\Pr(N = r)$ after the first i words of A,

$$q_0^{(0)} = 1, \qquad q_r^{(i)} \;=\; q_r^{(i-1)}\big(1 - p_i(\bar{t}_m)\big) \;+\; q_{r-1}^{(i-1)}\, p_i(\bar{t}_m),$$

and $\Pr(N \ge j \mid T = \bar{t}_m) = 1 - \sum_{r<j} q_r$. Counts above K are never used, so the recursion may be truncated there, costing $|A|K$ per cell rather than $|A|^2$.

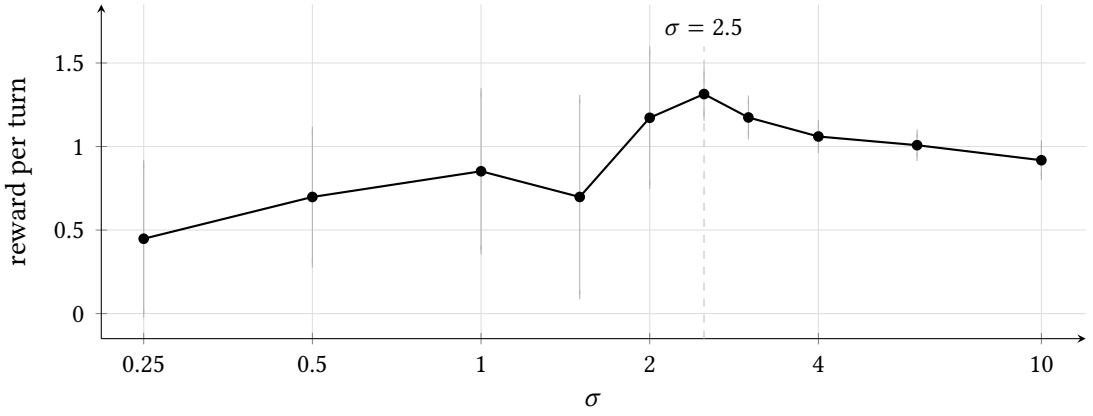
The total is $O\big(M(|A|K + |B|)\big)$ per clue, with M the number of cells. Every step is a fixed sequence of arithmetic on arrays of the same shape, so the whole clue vocabulary is scored at once rather than one clue at a time.

7 Results

The model above has one free parameter. To choose it, σ was swept over three orders of magnitude and each value was made to play the same 100 positions, drawn from held-out board words and spanning openings through endgames. Each turn was scored as the game scores it: the guesser's ranking is walked, own words are counted until the first word of B, and the reward is $+1$ per own word with -0.2, -1 and -10 for a neutral, opponent and assassin word respectively. The guesser was an external language model, not the similarity ranking the spymaster itself uses, so the numbers measure the model against a listener it has no access to.

σ	announced	revealed	gap	reward	s.e.	asn	opp	neu	clean
0.25	3.96	1.55	−2.41	0.448	0.228	5	53	36	6
0.5	3.91	1.60	−2.31	0.698	0.202	3	53	36	8
1.0	3.45	1.74	−1.71	0.852	0.241	4	43	29	24
1.5	2.79	1.72	−1.07	0.698	0.299	7	28	21	44
2.0	2.33	1.69	−0.64	1.172	0.205	3	19	14	64
2.5	1.72	1.41	−0.31	1.314	0.069	0	7	13	80
3.0	1.40	1.22	−0.18	1.174	0.054	0	3	8	89
4.0	1.21	1.10	−0.11	1.060	0.038	0	3	5	92
6.0	1.16	1.05	−0.11	1.008	0.035	0	3	6	91
10.0	1.13	0.99	−0.14	0.918	0.048	0	6	6	88

Per turn, over 100 positions. *announced* is k; *revealed* is the number of words of A the guesser actually took; the last four columns count how each turn ended.



Realized reward per turn. Bars are 95% confidence intervals; the wide bars at small σ are assassin turns, each costing -10.

Both limits saturate. As $\sigma \to 0$ the model treats the guesser as a copy of itself and announces $k = K$ on nearly every board, and as σ grows it announces $k = 1$ and stops responding to further increases. Neither end is optimal, and the reward curve is single-peaked between them.

Note that the number of words revealed and the reward earned are maximized at different σ. Revealed words peak at $\sigma = 1.0$ with 1.74 per turn, where reward is only 0.852; reward peaks at $\sigma = 2.5$ with 1.314, where 1.41 words are revealed. The difference is entirely risk. At $\sigma = 1.0$ the guesser reaches a word of B on 76 of 100 turns and the assassin on 4; at $\sigma = 2.5$ it reaches B on 20 and the assassin on none. Thus the gain from claiming an extra word is real but is more than repaid, and an objective fitted on revealed words alone would choose a value that loses games.

The assassin is what makes the left half of the curve hard to measure. It is worth -10, so a handful of assassin turns dominate the variance: the standard error is 0.2 to 0.3 for $\sigma \leq 2.0$ and 0.035 to 0.069 for $\sigma \geq 2.5$, where no assassin was reached at all. Comparing paired by board, $\sigma = 2.5$ beats every other value tested at the 5% level except $\sigma = 2.0$ ($+0.14 \pm 0.21$) and $\sigma = 1.0$ ($+0.46 \pm 0.49$), both of which are indistinguishable only because their own assassin losses are so noisy.

Note also that the gap between announced and revealed is negative everywhere and closes monotonically, reaching -0.11 at $\sigma = 4$. The model therefore remains optimistic at its own optimum, announcing 1.72 where 1.41 is delivered. Calibrating G and maximizing reward are different objectives: only the ordering of $G(k) - P(k)$ across clues and k affects the choice made, so a bias shared by every candidate leaves the argmax unchanged. Fitting σ to close the gap would report $\sigma \approx 4$ and cost 0.25 reward per turn.

We therefore take $\sigma = 2.5$. The positions used here are disjoint from the evaluation boards, so the value was not chosen on the boards it is later reported on.

Here the z_x are similarities standardized per clue, c_w are the costs of the words to avoid, and σ is the guesser's per-word perturbation.